

🧠 Generation Control Parameters

Day 4 — Notebook 3 of 4 | Estimated Time: 25 minutes

What You'll Learn

- How `temperature`, `top_k`, and `top_p` control randomness in generation
- When to use high vs. low temperature for different tasks
- How to produce deterministic (reproducible) outputs
- Practical experiments to build intuition for each parameter

Setup

```
%pip install -U -q "google-genai>=1.0.0"
```

```
!pip install google-auth==2.47.0
```

```
Requirement already satisfied: google-auth==2.47.0 in /usr/local/lib/python3.12/dist-packages (2.47.0)  
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (0.4.  
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (4.9.1)  
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->goog
```

```
import sys, os  
import time  
  
import json  
import re  
from google import genai  
from google.genai import types  
from IPython.display import Markdown  
  
try:  
    from google.colab import auth  
    auth.authenticate_user()  
    print("✅ Authenticated via Colab")  
except ImportError:  
    print("✅ Using local credentials (gcloud auth)")  
  
client = genai.Client(  
    vertexai=True,  
    project="chanakya-476010",  
    location="us-central1",  
)  
MODEL_ID = "gemini-2.5-flash"  
print("✅ Ready!")
```

```
✅ Authenticated via Colab  
✅ Ready!
```

1. How LLM Sampling Works

When an LLM generates a response, it doesn't pick the single "best" next word — it produces a **probability distribution** over all possible tokens and **samples** from it.

```
Prompt: "The capital of France is..."
```

```
Token probabilities:
```

```
"Paris"    → 92.3%  
"Lyon"     →  2.1%  
"London"   →  1.4%
```

```
"a" → 0.8%
... others → 3.4%
```

The three main parameters control **which tokens are eligible** and **how likely** the model is to pick lower-probability options:

Parameter	What it controls
temperature	Sharpness of the distribution (0 = deterministic, 2 = very random)
top_k	Maximum number of tokens to consider (e.g., top 40 only)
top_p	Cumulative probability threshold (e.g., only tokens summing to 95% probability)

2. Temperature

temperature controls how "creative" or "random" the output is.

- **temperature = 0** → nearly deterministic, always picks the most likely token
- **temperature = 0.7** → balanced: coherent but varied
- **temperature = 1.5+** → very creative, sometimes incoherent

```
prompt = "Continue this story in one sentence: 'The robot opened its eyes and saw'"
```

```
temperatures = [0.0, 0.5, 1.0, 1.5]
```

```
for temp in temperatures:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=temp),
    )
    print(f"🌡️ temperature={temp}: {response.text.strip()}")
    print()
```

```
🌡️ temperature=0.0: The robot opened its eyes and saw a field of bioluminescent flowers swaying gently under a twin-mooned sky
```

```
🌡️ temperature=0.5: The robot opened its eyes and saw a sprawling, neon-drenched metropolis shimmering under an alien sky.
```

```
🌡️ temperature=1.0: The robot opened its eyes and saw a sprawling, iridescent city stretching endlessly beneath a twin-sunned sky
```

```
🌡️ temperature=1.5: The robot opened its eyes and saw the shimmering expanse of a distant, star-filled nebula swirling with untold secrets
```

Observe:

- Low temperature → safe, predictable completions
- High temperature → unexpected, creative (sometimes strange) completions

3. Temperature for Reproducibility

For factual tasks (code generation, data extraction, classification), **low temperature** = more consistent results.

```
# Run the same factual question 3 times at high temperature
print("=== HIGH TEMPERATURE (1.5) - 3 runs ===")
for i in range(3):
    response = client.models.generate_content(
        model=MODEL_ID,
        contents="What is 17 multiplied by 24? Respond with just the number.",
        config=types.GenerateContentConfig(temperature=1.5),
    )
    print(f"Run {i+1}: {response.text.strip()}")

print()
print("=== LOW TEMPERATURE (0.0) - 3 runs ===")
for i in range(3):
    response = client.models.generate_content(
        model=MODEL_ID,
        contents="What is 17 multiplied by 24? Respond with just the number.",
        config=types.GenerateContentConfig(temperature=0.0),
    )
    print(f"Run {i+1}: {response.text.strip()}")
```

```

=== HIGH TEMPERATURE (1.5) - 3 runs ===
Run 1: 408
Run 2: 408
Run 3: 408

=== LOW TEMPERATURE (0.0) - 3 runs ===
Run 1: 408
Run 2: 408
Run 3: 408

```

4. top_p (Nucleus Sampling)

`top_p` filters the token pool to only the **most likely tokens** that together sum to probability `p`.

- **top_p = 1.0** → consider all tokens
- **top_p = 0.9** → only consider tokens that make up the top 90% of probability mass
- **top_p = 0.1** → very restricted — only the most probable tokens

This prevents the model from picking very unlikely "wild card" tokens while still allowing natural variety.

```

prompt = "Write a creative metaphor for the concept of 'time'. Just one sentence."

top_p_values = [0.1, 0.5, 0.9, 1.0]

for top_p in top_p_values:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=1.0, # Keep temperature constant to isolate top_p effect
            top_p=top_p,
        ),
    )
    print(f"🔥 top_p={top_p}: {response.text.strip()}")
    print()

```

```

🔥 top_p=0.1: Time is a silent alchemist, ceaselessly transmuting the raw material of the present into the gold of memory and th
🔥 top_p=0.5: Time is the invisible loom, perpetually weaving the intricate tapestry of existence from the ephemeral threads of
🔥 top_p=0.9: Time is the universe's patient sculptor, carving eternity into fleeting moments.
🔥 top_p=1.0: Time is the silent, unpausing loom, continuously weaving the present into the intricate fabric of the past.

```

5. top_k

`top_k` limits the token pool to the **top K most probable tokens** at each step.

- **top_k = 1** → always pick the single most likely token (greedy decoding)
- **top_k = 40** → choose from the 40 most likely tokens (Gemini default)
- **top_k = 1000** → very broad pool, close to unconstrained

```

prompt = "List 3 unusual uses for a paperclip."

top_k_values = [1, 10, 40, 60]

for top_k in top_k_values:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=1.0,
            top_k=top_k,
        ),
    )
    print(f"🧠 top_k={top_k}:")
    print(response.text.strip())
    print()

```

```

top_k=1:
Here are 3 unusual uses for a paperclip:

1.  **Improvised Stylus/Engraving Tool:** Unbend a paperclip and use the sharp point to engrave delicate lines into soft materia
2.  **Miniature Musical Instrument Tines:** Bend several paperclips into different shapes and lengths, then attach them to a sma
3.  **Emergency Fishing Lure/Hook:** In a survival situation, a paperclip can be bent and sharpened to create a tiny fishing hoo

top_k=10:
Here are 3 unusual uses for a paperclip:

1.  **Improvised Fishing Hook:** Bend it into a hook shape, tie some string or thread to it, and you have a makeshift hook for s
2.  **Splinter Remover:** Unbend a paperclip to create a fine, pointed tool. You can use this to carefully dig out stubborn spli
3.  **Nozzle Unclogger:** Straighten a paperclip and use the thin wire to clear blockages in spray bottle nozzles, glue dispense

top_k=40:
Here are 3 unusual uses for a paperclip:

1.  **Splinter Remover:** Straighten one end of the paperclip to a fine point. You can then use this point to carefully and gent
2.  **Zipper Pull Replacement:** If the pull tab on a zipper breaks off (on a jacket, bag, or jeans), you can loop a paperclip t
3.  **SIM Card Ejector Tool:** Most modern smartphones and tablets require a small, pointed tool to eject the SIM card tray. A s

top_k=60:
Here are 3 unusual uses for a paperclip:

1.  **Emergency Fishing Hook for Tiny Items:** Unbend it and shape it into a miniature hook to retrieve a dropped earring from a
2.  **Tiny Art Sculpture:** Bend and twist it into an abstract shape, a miniature person, an animal, or even a tiny piece of jew
3.  **Improvised Food Skewer/Toothpick:** In a pinch, a cleaned, unbent paperclip can be used to spear a small piece of food (li

```

6. max_output_tokens

Controls the **maximum length** of the response. Useful to prevent runaway outputs and manage cost.

```

for max_tokens in [20, 100, 500]:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents="Explain the theory of relativity.",
        config=types.GenerateContentConfig(
            max_output_tokens=max_tokens,
        ),
    )
    text = response.text or ""
    word_count = len(text.split())
    print(f" max_output_tokens={max_tokens} → ~{word_count} words")
    print(text.strip()[:200], "... " if len(text) > 200 else "")
    print()

 max_output_tokens=20 → ~0 words

 max_output_tokens=100 → ~2 words
The **Theory

 max_output_tokens=500 → ~16 words
The **Theory of Relativity** is a groundbreaking scientific theory developed by Albert Einstein, consisting of two

```

7. Practical Configuration Guide

Here are recommended settings for common use cases:

```

# Use case configurations
configs = {
    "factual_qa": types.GenerateContentConfig(
        temperature=0.1,
        top_p=0.9,
        top_k=40,
        max_output_tokens=512,
    ),
    "creative_writing": types.GenerateContentConfig(
        temperature=1.2,
        top_p=0.95,

```

```

        top_k=50,
        max_output_tokens=1024,
    ),
    "code_generation": types.GenerateContentConfig(
        temperature=0.2,
        top_p=0.95,
        top_k=40,
        max_output_tokens=2048,
    ),
    "brainstorming": types.GenerateContentConfig(
        temperature=1.0,
        top_p=1.0,
        top_k=64,
        max_output_tokens=512,
    ),
}

prompt = "Give me an idea for improving team productivity."

for use_case, config in configs.items():
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=prompt,
        config=config,
    )
    print(f"🌀 [{use_case}]\n")
    print(response.text.strip()[:300])
    print()

```

🌀 [factual_qa]

Okay, here's a comprehensive idea that tackles several common productivity blockers:

##

🌀 [creative_writing]

Here's an idea focused on **reducing context switching and improving communication clarity**, which are common drains on team pr

The Idea: **Implement "Focus Blocks" and "Asynchronous**

🌀 [code_generation]

Improving team productivity is a broad goal, so the best idea often depends on the specific challenges your team faces. However,

Here's a detailed idea:

Idea: Implement a "Communication & Mee

🌀 [brainstorming]

Okay, here's an idea that's often overlooked but can have a

🔧 Exercise 1: Temperature Tuning

Find the right temperature for a **customer support chatbot** that needs to be helpful but not too robotic or too unpredictable.

```

# Exercise 1: Temperature tuning for customer support
customer_message = """I ordered a laptop 2 weeks ago and it still hasn't arrived.
My order number is #ORD-4892. I'm really frustrated. What should I do?"""

system_instruction = f"""You are a helpful customer support agent for an e-commerce company.
Be empathetic, professional, and provide actionable steps. This is the customer message <source> {customer_message} </source>"""

# TODO: Try temperatures [0.0, 0.3, 0.7, 1.2] and decide which feels best for this use case
# Consider: Is the response empathetic? Is it helpful? Does it vary too much between runs?

temperatures_to_test = [0.0, 0.3, 0.7, 1.2]

for temp in temperatures_to_test:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=system_instruction,
        config=types.GenerateContentConfig(temperature=temp),
    )

```

```

print(f"🔥 temperature={temp}:\n\n {response.text.strip()}")
# TODO: Generate and print each response

```

```
🔥 temperature=0.0:
```

Oh no, I am so sorry to hear that your laptop order hasn't arrived yet, especially after two weeks. I completely understand how frustrating that must be. Thank you for providing your order number, #ORD-4892. I've noted it down.

Please allow me a moment to look into this for you right away. I will immediately check our shipping records, tracking information, and contact our shipping partner. I will get back to you with a comprehensive update and the next steps as soon as I have a clear picture, which should be within the hour. In the meantime, please rest assured that we are doing everything we can to locate your order and get it to you as quickly as possible. Thank you for your patience while I investigate this.

```
🔥 temperature=0.3:
```

Oh no, I completely understand how frustrating it must be to wait two weeks for your new laptop, especially when it hasn't arrived yet. Thank you for providing your order number, #ORD-4892. That's very helpful.

Please give me a moment while I look up your order details and tracking information. I'm going to investigate this thoroughly and get back to you as soon as I have an update. **Here's what I'll do and what you can expect:**

1. ****Immediate Investigation:**** I'm checking our system right now for the latest tracking updates and any internal notes regarding your order.
2. ****Carrier Contact (if needed):**** If the tracking information isn't clear, I will reach out directly to our shipping carrier to get more details.
3. ****Update You Promptly:**** I will reply to this message with a comprehensive update as soon as I have all the information, including:
 - * The current status of your shipment.
 - * The estimated delivery date.
 - * If there's been a delay, the reason for it and what steps we're taking to resolve it.
 - * If necessary, I'll discuss options for a replacement or further action.

Please bear with me for just a few minutes while I gather this information for you. We're committed to getting your laptop to you as quickly as possible. 🔥 temperature=0.7:

Oh no, I am so sorry to hear that your laptop hasn't arrived after two weeks. That's definitely frustrating, and I completely understand how you feel. Thank you for providing your order number, ****#ORD-4892****. That's very helpful!

Please allow me a moment to look into this for you right away. Here's what I'm going to do:

1. ****Immediately Check Tracking:**** I'll access our system to review the latest tracking information for your package.
2. ****Contact Shipping Partner (if needed):**** If the tracking isn't clear or shows an unexpected delay, I will reach out directly to our shipping partner.
3. ****Review Order Details:**** I'll double-check all the shipping information to ensure everything was processed correctly on our end.

There's nothing further you need to do at this very moment. I will personally investigate this thoroughly. I aim to get back to you with a clear update within the hour. Thank you for your patience as I work to get to the bottom of this. We'll make sure to get your laptop to you.

```
🔥 temperature=1.2:
```

Oh no, I completely understand how frustrating it must be to wait for your new laptop for two weeks, especially when you're excited to have it. Thank you for providing your order number, #ORD-4892. This is very helpful.

Please allow me a few moments to thoroughly investigate the status of your order. I will look into the tracking information, speak with our shipping partner, and get back to you as soon as I have a clear update. **Here's what I'll do and what you can expect:**

```
1. **Immediate Investigation:** I'm checking our system right now for the latest tracking updates and any internal notes regarding your order.
```

🔖 Exercise 2: Deterministic Classification

Build a sentiment classifier that gives **consistent results** across multiple runs.

```

# Exercise 2: Consistent sentiment classification
reviews = [
    "...This product exceeded all my expectations. Absolutely worth the price!",
    "...Terrible quality. Broke after two days. Complete waste of money.",
    "...It's okay. Does the job but nothing special.",
]

# TODO: Configure generation parameters so that:
# 1. The same review always gets the same classification across multiple runs
# 2. Output is constrained to only: POSITIVE, NEGATIVE, or NEUTRAL
# Hint: What temperature gives you the most consistent results?

```

```
# Run classification 3 times for each review and check consistency
for review in reviews:
    results = []
    for _ in range(3):
        response = client.models.generate_content(
            model=MODEL_ID,
            contents=f"Classify the review as Positive / Negative / Neutral given this is the source of information <source> {review}
            config=types.GenerateContentConfig(temperature=0.0))
        print(f"Review: {review[:50]}... → Results: {response.text}")
```

Review: This product exceeded all my expectations. Absolut... → Results: Positive
Review: This product exceeded all my expectations. Absolut... → Results: Positive
Review: This product exceeded all my expectations. Absolut... → Results: Positive
Review: Terrible quality. Broke after two days. Complete w... → Results: Negative
Review: Terrible quality. Broke after two days. Complete w... → Results: Negative
Review: Terrible quality. Broke after two days. Complete w... → Results: Negative
Review: It's okay. Does the job but nothing special.... → Results: Neutral
Review: It's okay. Does the job but nothing special.... → Results: Neutral
Review: It's okay. Does the job but nothing special.... → Results: Neutral

Parameter Quick Reference

Parameter	Range	Low Value	High Value	Best for
temperature	0.0–2.0	Deterministic	Creative/random	Low: classification, code, facts. High: creative writing, brainstorming
top_p	0.0–1.0	Restrictive	All tokens	0.9–0.95 for most tasks
top_k	1–1000	Greedy	Very open	40 is a good default
max_output_tokens	1–8192+	Short	Long	Match expected output length

Further Reading

Resource	Type	Link
Gemini API Generation Config	Docs	ai.google.dev/gemini-api/docs/text-generation
Temperature in LLMs Explained	Blog	huggingface.co/blog/how-to-generate

Next up: [04_Safety_Settings_and_Guardrails.ipynb](#)